Java Grundlagen

Datentypen, Operatoren, Kontrollstrukturen und Methoden

Autor

Luis Staudt

19. Dezember 2025

Inhaltsverzeichnis

Inhaltsverzeichnis . . . 2
Java Datentypen . . . 4
Primitive Datentypen . . . 4
int - Ganzzahlen . . . 5
double - Fließkommazahlen . . . 7
boolean - Wahrheitswerte . . . 9
char - Einzelnes Zeichen . . . 11
byte - Sehr kleine Ganzzahlen . . . 13
short - Kleine Ganzzahlen . . . 14
long - Sehr große Ganzzahlen . . . 15
float - Einfache Fließkommazahlen . . . 16
String - Zeichenketten . . . 17
Typkonvertierung . . . 20
Übersicht: Primitive Datentypen . . . 22
Arrays . . . 22
array.length - Länge eines Arrays . . . 24
array[index] - Element lesen . . . 26
array[index] = wert - Element schreiben . . . 28
Häufige Fehler . . . 30
Operatoren in Java . . . 33
Arithmetische Operatoren . . . 33
Vergleichsoperatoren . . . 34
Logische Operatoren . . . 36
Zuweisungsoperatoren . . . 38
Inkrement- und Dekrement-Operatoren . . . 39
String-Konkatenation . . . 40
Operatorrangfolge (Priorität) . . . 41
Kontrollstrukturen in Java . . . 43
If-Bedingungen . . . 43
For-Schleifen . . . 49
While-Schleifen . . . 57
Vergleich: For vs. While . . . 60
Häufige Fehler . . . 61

Grundlegende String-Methoden in Java 63
String-Methoden 63
Häufige Fehler bei Strings 65

Java Datentypen

Primitive Datentypen

Java unterscheidet zwischen primitiven Datentypen und Referenztypen (Objekten). Primitive Datentypen sind die grundlegenden Bausteine und speichern direkt Werte.

int - Ganzzahlen

Was ist int?

Der Datentyp `int` (Integer) speichert ganze Zahlen ohne Nachkommastellen.

Wertebereich:

- Von: -2.147.483.648
- Bis: 2.147.483.647
- Speichergröße: 32 Bit (4 Bytes)

Syntax:

```
int variablenName = wert;
```

Beispiele:

```
// Deklaration und Initialisierung
int alter = 25;
int temperatur = -5;
int punkte = 0;

// Ohne Initialisierung
int zahl;
zahl = 42;

// Berechnungen
int summe = 10 + 20;          // summe = 30
int differenz = 50 - 15;      // differenz = 35
int produkt = 6 * 7;          // produkt = 42
int quotient = 20 / 4;        // quotient = 5
int rest = 17 % 5;            // rest = 2

// Mit Variablen rechnen
int a = 10;
int b = 3;
int ergebnis = a + b;         // ergebnis = 13

// Inkrement und Dekrement
int zaehler = 5;
zaehler++;                    // zaehler = 6
zaehler--;                    // zaehler = 5

```

```
// Zusammengesetzte Operatoren
int wert = 10;
wert += 5;                    // wert = 15 (entspricht: wert = wert + 5)
wert -= 3;                    // wert = 12
wert *= 2;                    // wert = 24
wert /= 4;                    // wert = 6
```

Worauf achten:

- Division von int durch int ergibt int: `5 / 2 = 2` (nicht 2.5!)
- Bei Überschreitung des Wertebereichs: **Overflow** (Wert springt um)
- Standard-Wert bei Arrays: `0`
- Keine Nachkommastellen möglich

double - Fließkommazahlen

Was ist double?

Der Datentyp `double` speichert Zahlen mit Nachkommastellen (Gleitkommazahlen mit doppelter Genauigkeit).

Wertebereich:

- Sehr große und sehr kleine Zahlen mit ca. 15 Dezimalstellen Genauigkeit
- Von: $\pm 4.9 \times 10^{-324}$
- Bis: $\pm 1.7 \times 10^{308}$
- Speichergröße: 64 Bit (8 Bytes)

Syntax:

```
double variablenName = wert;
```

Beispiele:

```
// Deklaration und Initialisierung
double pi = 3.14159;
double temperatur = -5.5;
double gewicht = 75.8;
double ergebnis = 0.0;

// Berechnungen
double summe = 10.5 + 20.3;        // summe = 30.8
double produkt = 2.5 * 4.0;        // produkt = 10.0
double quotient = 7.0 / 2.0;       // quotient = 3.5

// Wichtig: Division mit Nachkommastellen
double a = 5.0;
double b = 2.0;
double teil = a / b;               // teil = 2.5

// int zu double konvertieren
int ganzzahl = 5;
double kommazahl = ganzzahl / 2.0; // kommazahl = 2.5
// ODER
double kommazahl2 = (double) ganzzahl / 2; // kommazahl2 = 2.5

// Wissenschaftliche Notation
```

```
double klein = 1.5e-4;              // = 0.00015
double gross = 3.2e8;               // = 320000000.0

// Mathematische Funktionen
double wurzel = Math.sqrt(16.0);    // wurzel = 4.0
double potenz = Math.pow(2, 3);     // potenz = 8.0
double absolut = Math.abs(-5.5);    // absolut = 5.5
```

Worauf achten:

- Punkt als Dezimaltrennzeichen, **nicht** Komma: `3.14` nicht `3,14`
- Rundungsfehler möglich: `0.1 + 0.2 = 0.30000000000000004`
- Division `5 / 2` ergibt `2`, aber `5.0 / 2.0` ergibt `2.5`
- Standard-Wert bei Arrays: `0.0`
- Für Geldbeträge besser `BigDecimal` verwenden

boolean - Wahrheitswerte

Was ist boolean?

Der Datentyp `boolean` speichert Wahrheitswerte: entweder `true` (wahr) oder `false` (falsch).

Wertebereich:

- Nur zwei mögliche Werte: `true` oder `false`
- Speichergröße: 1 Bit (theoretisch), praktisch abhängig von JVM

Syntax:

```
boolean variablenName = true/false;
```

Beispiele:

```
// Deklaration und Initialisierung
boolean istWahr = true;
boolean istFalsch = false;
boolean hatBestanden = true;

// Vergleichsoperatoren ergeben boolean
int alter = 18;
boolean istErwachsen = (alter >= 18);      // true
boolean istKind = (alter < 12);            // false
boolean istGenau18 = (alter == 18);        // true
boolean istNicht18 = (alter != 18);        // false

// Logische Operatoren
boolean a = true;
boolean b = false;

boolean und = a && b;                      // false (beide muessen true
    sein)
boolean oder = a || b;                     // true (mindestens einer true)
boolean nicht = !a;                        // false (Negation)

// In Bedingungen
int punkte = 85;
boolean bestanden = punkte >= 50;
if (bestanden) {
    System.out.println("Bestanden!");
}
```

```java

// Kombinierte Bedingungen
int temperatur = 25;
boolean angenehm = temperatur >= 18 && temperatur <= 28;

// Boolean in Schleifen
boolean weitermachen = true;
while (weitermachen) {
    // Code
    weitermachen = false;  // Schleife beenden
}

// Vergleich mit Strings
String name = "Max";
boolean istMax = name.equals("Max");       // true
```

Worauf achten:

- Nur `true` oder `false`, **keine** Zahlen (nicht wie in C: 0 = false, 1 = true)
- Kleinschreibung: `true`/`false`, nicht `True`/`False`
- Standard-Wert bei Arrays: `false`
- `==` für primitive Typen, `.equals()` für Objekte (z.B. Strings)

char - Einzelnes Zeichen

Was ist char?

Der Datentyp `char` speichert ein einzelnes Zeichen (Character) im Unicode-Format.

Wertebereich:

- Alle Unicode-Zeichen (Buchstaben, Ziffern, Sonderzeichen)
- Von: 0 bis 65.535 (als Zahlenwert)
- Speichergröße: 16 Bit (2 Bytes)

Syntax:

```
char variablenName = 'Zeichen';  // EINFACHE Anfuehrungszeichen!
```

Beispiele:

```
// Deklaration und Initialisierung
char buchstabe = 'A';
char ziffer = '5';
char sonderzeichen = '!';
char leerzeichen = ' ';

// Mit Unicode-Werten
char euro = '\u20AC';                      //    -Symbol
char herzchen = '\u2764';                  //

// Escape-Sequenzen
char neuzeile = '\n';                      // Zeilenumbruch
char tab = '\t';                           // Tabulator
char backslash = '\\';                     // \
char anfuehrung = '\'';                    // '

// Zeichen vergleichen
char note = 'A';
if (note == 'A') {
    System.out.println("Sehr gut!");
}

// Zeichen in Berechnungen (ASCII/Unicode-Werte)
char a = 'A';                              // Unicode-Wert: 65
char b = 'B';                              // Unicode-Wert: 66
int differenz = b - a;                     // differenz = 1
```

```java

// Kleinbuchstabe zu Gro buchstabe
char klein = 'a';                          // Unicode-Wert: 97
char gross = (char) (klein - 32);          // gross = 'A' (Wert: 65)

// Pr fen ob Buchstabe
char zeichen = 'M';
boolean istBuchstabe = (zeichen >= 'A' && zeichen <= 'Z') ||
                       (zeichen >= 'a' && zeichen <= 'z');

// Pr fen ob Ziffer
char z = '7';
boolean istZiffer = (z >= '0' && z <= '9');

// char aus String extrahieren
String text = "Hallo";
char erstes = text.charAt(0);              // erstes = 'H'
```

Worauf achten:

- **Einfache** Anführungszeichen: `'A'`, nicht Ä“
- Nur **ein** Zeichen: `'A'` ist char, Ä“ ist String
- `char` ist **nicht** dasselbe wie `String`
- Standard-Wert bei Arrays: `'\u0000'` (Null-Zeichen)
- Kann als Zahlenwert verwendet werden (Unicode-Wert)

byte - Sehr kleine Ganzzahlen

Was ist byte?

Der Datentyp `byte` speichert sehr kleine ganze Zahlen. Wird verwendet, wenn Speicherplatz wichtig ist.

Wertebereich:

- Von: -128
- Bis: 127
- Speichergröße: 8 Bit (1 Byte)

Syntax:

```
byte variablenName = wert;
```

Beispiele:

```
// Deklaration und Initialisierung
byte alter = 25;
byte temperatur = -15;
byte prozent = 100;

// H ufig bei Datei-Operationen
byte[] daten = new byte[1024];

// Konvertierung von int zu byte
int gross = 100;
byte klein = (byte) gross;

// Vorsicht bei  berschreitung
byte max = 127;
byte ueberlauf = (byte) (max + 1);         // ueberlauf = -128 (Overflow!)
```

Worauf achten:

- Sehr kleiner Wertebereich: nur -128 bis 127
- Bei Überschreitung: Overflow
- Wird oft für Binärdaten verwendet
- Standard-Wert bei Arrays: `0`

short - Kleine Ganzzahlen

Was ist short?

Der Datentyp `short` speichert kleine ganze Zahlen. Ein Mittelweg zwischen `byte` und `int`.

Wertebereich:

- Von: -32.768
- Bis: 32.767
- Speichergröße: 16 Bit (2 Bytes)

Syntax:

```
short variablenName = wert;
```

Beispiele:

```
// Deklaration und Initialisierung
short jahr = 2024;
short distanz = 5000;

// Konvertierung
int gross = 1000;
short klein = (short) gross;
```

Worauf achten:

- Wird selten verwendet, meist wird direkt `int` genommen
- Wertebereich: -32.768 bis 32.767
- Standard-Wert bei Arrays: `0`

long - Sehr große Ganzzahlen

Was ist long?

Der Datentyp `long` speichert sehr große ganze Zahlen, die über den `int`-Bereich hinausgehen.

Wertebereich:

- Von: -9.223.372.036.854.775.808
- Bis: 9.223.372.036.854.775.807
- Speichergröße: 64 Bit (8 Bytes)

Syntax:

```
long variablenName = wert;
long grosseZahl = wertL;  // L am Ende fuer long-Literal
```

Beispiele:

```
// Deklaration und Initialisierung
long weltbevoelkerung = 8000000000L;       // L am Ende!
long millisekunden = System.currentTimeMillis();

// Ohne L wird es als int interpretiert (Fehler bei gro en Zahlen)
// long falsch = 8000000000;               // FEHLER!
long richtig = 8000000000L;                // Korrekt

// Berechnungen
long summe = 1000000000L + 2000000000L;
```

Worauf achten:

- Bei großen Zahlen: `L` am Ende anhängen: `1000000000L`
- Ohne `L` wird Zahl als `int` interpretiert (kann zu Fehlern führen)
- Standard-Wert bei Arrays: `0L`
- Verwenden, wenn Zahlen größer als ca. 2 Milliarden sind

float - Einfache Fließkommazahlen

Was ist float?

Der Datentyp `float` speichert Fließkommazahlen mit einfacher Genauigkeit. Weniger genau als `double`.

Wertebereich:

- Ca. 6-7 Dezimalstellen Genauigkeit
- Von: $\pm 1.4 \times 10^{-45}$
- Bis: $\pm 3.4 \times 10^{38}$
- Speichergröße: 32 Bit (4 Bytes)

Syntax:

```
float variablenName = wertF;  // F am Ende fuer float-Literal
```

Beispiele:

```
// Deklaration und Initialisierung
float pi = 3.14F;                          // F am Ende!
float temperatur = 36.5F;
float gewicht = 75.8F;

// Ohne F wird es als double interpretiert
// float falsch = 3.14;                    // Warnung/Fehler
float richtig = 3.14F;                     // Korrekt

// Berechnungen
float summe = 10.5F + 20.3F;
```

Worauf achten:

- `F` oder `f` am Ende anhängen: `3.14F`
- Weniger genau als `double`
- In der Praxis wird meist `double` bevorzugt
- Standard-Wert bei Arrays: `0.0F`

String - Zeichenketten

Was ist ein String?

Beschreibung

Ein `String` ist **kein primitiver Datentyp**, sondern eine Klasse (Referenztyp). Er speichert Zeichenketten (Text).

Eigenschaften:

- Unveränderlich (immutable): Einmal erstellt, kann der Inhalt nicht geändert werden
- Wird mit **doppelten** Anführungszeichen erstellt: `TText“`
- Kann beliebig lang sein

Syntax:

```
String variablenName = "Text";
```

Beispiele:

```
// Deklaration und Initialisierung
String name = "Max Mustermann";
String begruessung = "Hallo Welt!";
String leer = "";                           // Leerer String
String mitZahl = "Ich bin 25 Jahre alt";

// String-Konkatenation (Verkettung)
String vorname = "Max";
String nachname = "Mustermann";
String vollername = vorname + " " + nachname;  // "Max Mustermann"

// Mit Zahlen verketten
int alter = 25;
String text = "Ich bin " + alter + " Jahre alt";

// String-Methoden
String wort = "Hallo";
int laenge = wort.length();                 // 5
char erstes = wort.charAt(0);               // 'H'
String gross = wort.toUpperCase();          // "HALLO"
String klein = wort.toLowerCase();          // "hallo"

// Strings vergleichen
```

```java
String a = "Hallo";
String b = "Hallo";
boolean gleich = a.equals(b);              // true
boolean gleichIgnoreCase = a.equalsIgnoreCase("HALLO");  // true

// WICHTIG: Nicht mit == vergleichen!
String s1 = "Test";
String s2 = new String("Test");
boolean falsch = (s1 == s2);               // false (vergleicht Referenzen)
boolean richtig = s1.equals(s2);           // true (vergleicht Inhalt)

// Teilstrings
String text2 = "Hallo Welt";
String teil = text2.substring(0, 5);       // "Hallo"
String ab = text2.substring(6);            // "Welt"

// Suchen in Strings
String satz = "Java ist toll";
boolean enthaelt = satz.contains("ist");   // true
int position = satz.indexOf("ist");        // 5
boolean beginnt = satz.startsWith("Java"); // true
boolean endet = satz.endsWith("toll");     // true

// Ersetzen
String original = "Hallo Welt";
String neu = original.replace("Welt", "Java");  // "Hallo Java"

// Leerzeichen entfernen
String mitLeer = "  Text  ";
String ohneLeer = mitLeer.trim();          // "Text"

// String in Array umwandeln
String liste = "Apfel,Birne,Orange";
String[] fruechte = liste.split(",");      // ["Apfel", "Birne", "Orange"]

// Zahlen zu String
int zahl = 42;
String zahlString = String.valueOf(zahl);  // "42"
// ODER
String zahlString2 = "" + zahl;            // "42"

// String zu Zahl
String nummerText = "123";
int nummer = Integer.parseInt(nummerText); // 123
double komma = Double.parseDouble("3.14"); // 3.14
```

Worauf achten:

- **Doppelte** Anführungszeichen: `TText“`, nicht `'Text'`
- Strings mit `.equals()` vergleichen, **nicht** mit `==`
- `String` mit großem **S** schreiben
- Strings sind unveränderlich: Methoden geben neuen String zurück
- `null` ist möglich, aber (leerer String) ist etwas anderes
- Standard-Wert bei Arrays: `null`

Typkonvertierung

Implizite Konvertierung (Automatisch)

Beschreibung

Java konvertiert automatisch von “kleineremßu “größerem"Datentyp ohne Datenverlust.

Beispiele:

```
// int zu double (automatisch)
int ganzzahl = 5;
double kommazahl = ganzzahl;                // kommazahl = 5.0

// byte zu int (automatisch)
byte klein = 100;
int gross = klein;                          // gross = 100

// char zu int (automatisch)
char buchstabe = 'A';
int code = buchstabe;                       // code = 65 (Unicode-Wert)

// In Berechnungen
int a = 5;
double b = 2.5;
double ergebnis = a + b;                    // ergebnis = 7.5
```

Explizite Konvertierung (Type Cast)

Beschreibung

Von “größeremßu “kleinerem"Datentyp muss explizit gecastet werden. Kann zu Datenverlust führen!

Syntax:

```
zielTyp variable = (zielTyp) quellVariable;
```

Beispiele:

```
// double zu int (Nachkommastellen gehen verloren!)
double kommazahl = 3.7;
int ganzzahl = (int) kommazahl;             // ganzzahl = 3 (nicht 4!)

// long zu int
```

```
long gross = 1000L;
int klein = (int) gross;

// int zu byte (Vorsicht bei gro en Zahlen!)
int zahl = 200;
byte kleinByte = (byte) zahl;              // Overflow!

// Division mit Cast
int a = 5;
int b = 2;
double ergebnis = (double) a / b;          // ergebnis = 2.5

// Runden statt Abschneiden
double wert = 3.7;
int gerundet = (int) Math.round(wert);     // gerundet = 4
```

Worauf achten:

- Cast von double zu int: Nachkommastellen werden **abgeschnitten**, nicht gerundet!
- Bei Überschreitung: Datenverlust oder unerwartete Werte
- Für Rundung: `Math.round()` verwenden
- Implizite Konvertierung: klein → groß
- Explizite Konvertierung (Cast): groß → klein

Übersicht: Primitive Datentypen

Typ	Größe	Wertebereich	Standard
byte	8 Bit	-128 bis 127	0
short	16 Bit	-32.768 bis 32.767	0
int	32 Bit	-2.147.483.648 bis 2.147.483.647	0
long	64 Bit	ca. -9,2 $\times$ 10^{18} bis 9,2 $\times$ 10^{18}	0L
float	32 Bit	ca. $\pm$3,4 $\times$ 10^{38}	0.0F
double	64 Bit	ca. $\pm$1,7 $\times$ 10^{308}	0.0
char	16 Bit	0 bis 65.535 (Unicode)	'\u0000'
boolean	1 Bit	true oder false	false

Tabelle 1: Übersicht der primitiven Datentypen in Java

Arrays

Was ist ein Array?

Beschreibung

Ein Array ist eine Datenstruktur, die eine **feste Anzahl** von Elementen des **gleichen Datentyps** speichert. Arrays sind Referenztypen (wie Strings), keine primitiven Datentypen.

Eigenschaften:

- Feste Größe: Nach der Erstellung kann die Größe nicht mehr geändert werden
- Index beginnt bei 0: Erstes Element hat Index 0, letztes Element hat Index `length - 1`
- Alle Elemente müssen den gleichen Typ haben
- Standard-Werte: Arrays werden automatisch mit Standard-Werten initialisiert (0, false, null, etc.)

Syntax:

```
// Deklaration und Initialisierung mit G r   e
datentyp[] arrayName = new datentyp[groesse];

// Deklaration und Initialisierung mit Werten
datentyp[] arrayName = {wert1, wert2, wert3};
```

Beispiele:

```java
// Array mit fester G r   e erstellen
int[] zahlen = new int[5];        // 5 Pl tze, alle 0
String[] namen = new String[3];   // 3 Pl tze, alle null
boolean[] flags = new boolean[4]; // 4 Pl tze, alle false

// Array mit Werten erstellen
int[] noten = {1, 2, 3, 2, 1};
String[] wochentage = {"Mo", "Di", "Mi", "Do", "Fr"};
double[] preise = {9.99, 14.50, 7.25};

// Mehrdimensionale Arrays
int[][] matrix = new int[3][3];   // 3x3 Matrix
int[][] tabelle = {{1, 2}, {3, 4}, {5, 6}};
```

Worauf achten:

- Größe ist **fix** nach Erstellung
- Index beginnt bei **0**, nicht bei 1
- Alle Elemente müssen den **gleichen Typ** haben
- Standard-Wert bei Array-Variablen: `null`

array.length - Länge eines Arrays

Was ist array.length?

`length` ist ein **Attribut** (keine Methode!), das die Anzahl der Elemente im Array zurückgibt.

Eigenschaften:

- Ist ein Attribut, keine Methode: **ohne** Klammern `()`
- Gibt die Größe zurück, mit der das Array erstellt wurde
- Funktioniert auch bei leeren Arrays (nur Standard-Werte)

Syntax:

```
int laenge = arrayName.length;  // OHNE Klammern!
```

Beispiele:

```
// L nge verschiedener Arrays
int[] zahlen = {10, 20, 30, 40, 50};
int anzahl = zahlen.length;                // anzahl = 5

String[] namen = new String[10];
int groesse = namen.length;                // groesse = 10 (auch wenn leer!)

// Leeres Array
int[] leer = new int[0];
int laenge = leer.length;                  // laenge = 0

// In Schleife verwenden
int[] werte = {1, 2, 3, 4};
for (int i = 0; i < werte.length; i++) {
    System.out.println(werte[i]);
}

// Letztes Element zugreifen
int[] liste = {100, 200, 300};
int letzter = liste[liste.length - 1];   // letzter = 300

// Bei mehrdimensionalen Arrays
int[][] matrix = new int[3][4];
int zeilen = matrix.length;                // zeilen = 3
int spalten = matrix[0].length;            // spalten = 4
```

Worauf achten:

- `.length` **ohne Klammern** (bei Arrays)
- `.length()` **mit Klammern** (bei Strings)
- Gibt die Größe zurück, mit der das Array erstellt wurde
- Auch bei lleerenÄrrays (nur `null`-Werte) ist die Länge die Anzahl der Plätze

array[index] - Element lesen

Was ist array[index]?

Mit der eckigen Klammer `[]` greift man auf ein einzelnes Element im Array zu. Der Index gibt die Position an.

Eigenschaften:

- Index beginnt bei 0: Erstes Element ist `array[0]`
- Letztes Element ist `array[array.length - 1]`
- Bei ungültigem Index: `ArrayIndexOutOfBoundsException`

Syntax:

```
datentyp element = arrayName[index];
```

Beispiele:

```
// Einzelne Elemente lesen
int[] zahlen = {10, 20, 30, 40, 50};
int erstes = zahlen[0];                    // erstes = 10
int zweites = zahlen[1];                   // zweites = 20
int letztes = zahlen[4];                   // letztes = 50

// Alle Elemente durchlaufen
String[] woerter = {"Hallo", "Welt", "Java"};
for (int i = 0; i < woerter.length; i++) {
    String wort = woerter[i];
    System.out.println(wort);
}

// Bestimmte Position pr fen
int[] noten = {1, 2, 3, 2, 1};
if (noten[2] == 3) {
    System.out.println("Dritte Note ist eine 3");
}

// Letztes Element
int[] liste = {5, 10, 15, 20};
int letzterWert = liste[liste.length - 1];  // letzterWert = 20

// Element in Berechnung verwenden
int[] preise = {100, 200, 150};
```

```
26 int summe = preise[0] + preise[1] + preise[2];  // summe = 450
27
28 // Mehrdimensionale Arrays
29 int[][] matrix = {{1, 2, 3}, {4, 5, 6}};
30 int wert = matrix[0][1];                  // wert = 2 (1. Zeile, 2. Spalte)
```

Worauf achten:

- Index beginnt bei **0**, nicht bei 1
- Index muss zwischen 0 und `array.length - 1` liegen
- Bei ungültigem Index: `ArrayIndexOutOfBoundsException`
- Letztes Element: `array[array.length - 1]`

array[index] = wert - Element schreiben

Was ist array[index] = wert?

Mit der Zuweisung kann man den Wert eines Elements an einer bestimmten Position im Array ändern.

Eigenschaften:

- Überschreibt den vorherigen Wert an dieser Position
- Der neue Wert muss zum Datentyp des Arrays passen
- Index muss gültig sein (0 bis `length - 1`)

Syntax:

```
arrayName[index] = neuerWert;
```

Beispiele:

```
// Array bef llen
int[] zahlen = new int[5];                // Array mit 5 Pl tzen, alle 0
zahlen[0] = 10;                           // Erstes Element auf 10 setzen
zahlen[1] = 20;                           // Zweites Element auf 20 setzen
zahlen[4] = 50;                           // Letztes Element auf 50 setzen
// Ergebnis: {10, 20, 0, 0, 50}

// Werte  ndern
String[] namen = {"Anna", "Bob", "Clara"};
namen[1] = "Max";                         // "Bob" wird zu "Max"
// Ergebnis: {"Anna", "Max", "Clara"}

// In Schleife bef llen
int[] quadrate = new int[5];
for (int i = 0; i < quadrate.length; i++) {
    quadrate[i] = i * i;
}
// Ergebnis: {0, 1, 4, 9, 16}

// Wert erh hen
int[] punkte = {10, 20, 30};
punkte[0] = punkte[0] + 5;                // Erstes Element um 5 erh hen
// ODER k rzer:
punkte[0] += 5;
// Ergebnis: {15, 20, 30}
```

```java

// Array "leeren" (auf Standard-Wert setzen)
int[] werte = {1, 2, 3, 4, 5};
for (int i = 0; i < werte.length; i++) {
    werte[i] = 0;
}
// Ergebnis: {0, 0, 0, 0, 0}

// Zwei Elemente tauschen
int[] liste = {10, 20, 30};
int temp = liste[0];                        // Zwischenspeichern
liste[0] = liste[2];                        // 30 nach vorne
liste[2] = temp;                            // 10 nach hinten
// Ergebnis: {30, 20, 10}

// Array aus Benutzereingabe f llen
int[] zahlenListe = new int[3];
for (int i = 0; i < zahlenListe.length; i++) {
    // zahlenListe[i] = scanner.nextInt();
}

// Mehrdimensionale Arrays
int[][] matrix = new int[2][2];
matrix[0][0] = 1;
matrix[0][1] = 2;
matrix[1][0] = 3;
matrix[1][1] = 4;
// Ergebnis: {{1, 2}, {3, 4}}
```

Worauf achten:

- Index muss zwischen 0 und `array.length - 1` liegen
- Bei ungültigem Index: `ArrayIndexOutOfBoundsException`
- Der Wert muss zum Datentyp des Arrays passen
- Überschreibt den vorherigen Wert an dieser Position komplett

Häufige Fehler

String mit == vergleichen

```
// FALSCH
String a = "Hallo";
String b = "Hallo";
if (a == b) { }  // Vergleicht Referenzen, nicht Inhalt!

// RICHTIG
if (a.equals(b)) { }  // Vergleicht Inhalt
```

Integer-Division

```
// FALSCH - ergibt 2, nicht 2.5
int a = 5;
int b = 2;
double ergebnis = a / b;  // ergebnis = 2.0

// RICHTIG
double ergebnis = (double) a / b;  // ergebnis = 2.5
// ODER
double ergebnis = 5.0 / 2.0;  // ergebnis = 2.5
```

char vs. String verwechseln

```
// FALSCH
char buchstabe = "A";  // Fehler: doppelte Anfuehrungszeichen
String wort = 'Hallo';  // Fehler: einfache Anfuehrungszeichen

// RICHTIG
char buchstabe = 'A';   // einfache Anfuehrungszeichen
String wort = "Hallo";  // doppelte Anfuehrungszeichen
```

L und F vergessen

```
// FALSCH
long gross = 5000000000;  // Fehler: zu gross fuer int
float zahl = 3.14;        // Warnung: double wird zu float

// RICHTIG
long gross = 5000000000L;  // L am Ende
float zahl = 3.14F;        // F am Ende
```

Overflow nicht beachten

```
// Problem
byte b = 127;
b++;  // b = -128 (Overflow!)

int i = 2147483647;
i++;  // i = -2147483648 (Overflow!)
```

null vs. leerer String

```
String a = null;      // Keine Referenz (keine Methoden aufrufbar!)
String b = "";        // Leerer String (Methoden aufrufbar)

// a.length();        // FEHLER: NullPointerException
int laenge = b.length();  // Funktioniert: laenge = 0
```

String vs. Array - length

```
// FALSCH
String text = "Hallo";
int laenge = text.length;  // Fehler: Strings brauchen ()

int[] zahlen = {1, 2, 3};
int anzahl = zahlen.length();  // Fehler: Arrays haben KEINE ()

// RICHTIG
int laenge = text.length();    // String MIT Klammern
int anzahl = zahlen.length;    // Array OHNE Klammern
```

Array-Index-Grenzen

```
// FALSCH
int[] zahlen = {10, 20, 30};  // length = 3, Indizes: 0, 1, 2
int x = zahlen[3];            // Fehler! Index 3 existiert nicht
int y = zahlen[-1];           // Fehler! Negative Indizes nicht erlaubt

// RICHTIG
int letzter = zahlen[zahlen.length - 1];  // Index 2
int erster = zahlen[0];                   // Index 0
```

Array-Größe nach Erstellung ändern

```
// FALSCH
int[] zahlen = new int[5];
zahlen.length = 10;  // Fehler! length ist read-only

// RICHTIG - neues Array erstellen und Werte kopieren
int[] zahlen = new int[5];
int[] neuesArray = new int[10];
for (int i = 0; i < zahlen.length; i++) {
    neuesArray[i] = zahlen[i];
}
zahlen = neuesArray;  // Referenz auf neues Array
```

Operatoren in Java

Arithmetische Operatoren

Beschreibung:

Werden für mathematische Berechnungen verwendet.

Operator	Bedeutung	Beispiel	Ergebnis
+	Addition	5 + 3	8
-	Subtraktion	5 - 3	2
*	Multiplikation	5 * 3	15
/	Division	10 / 3	3 (bei int)
%	Modulo (Rest)	10 % 3	1

Beispiele:

```
int a = 10;
int b = 3;
int summe = a + b;          // summe = 13
int differenz = a - b;      // differenz = 7
int produkt = a * b;        // produkt = 30
int quotient = a / b;       // quotient = 3 (Ganzzahldivision!)
int rest = a % b;           // rest = 1

// Gerade oder ungerade pruefen
int zahl = 7;
if (zahl % 2 == 0) {
    System.out.println("Gerade");
} else {
    System.out.println("Ungerade");
}
```

Wichtig bei Division:

```
int x = 7 / 2;       // x = 3 (int-Division schneidet ab)
double y = 7.0 / 2; // y = 3.5 (mit Kommazahl)
```

Vergleichsoperatoren

Beschreibung:

Vergleichen zwei Werte und geben `true` oder `false` zurück.

Operator	Bedeutung	Beispiel	Ergebnis
==	Gleich	`5 == 5`	`true`
!=	Ungleich	`5 != 3`	`true`
>	Größer	`5 > 3`	`true`
<	Kleiner	`5 < 3`	`false`
>=	Größer oder gleich	`5 >= 5`	`true`
<=	Kleiner oder gleich	`5 <= 3`	`false`

Beispiele:

```
int a = 10;
int b = 5;

if (a > b) {
    System.out.println("a ist groesser");
}

if (a == 10) {
    System.out.println("a ist 10");
}

if (b != 0) {
    int ergebnis = a / b;  // Division nur wenn b nicht 0 ist
}

// Array-Index pruefen
int[] zahlen = {1, 2, 3};
int index = 2;
if (index >= 0 && index < zahlen.length) {
    System.out.println(zahlen[index]);
}
```

Wichtig bei Strings:

```
String text1 = "Hallo";
String text2 = "Hallo";

// FALSCH - vergleicht Objektreferenzen, nicht Inhalt!
if (text1 == text2) { }
```

```
6
7 // RICHTIG - vergleicht Inhalt
8 if (text1.equals(text2)) { }
```

Logische Operatoren

Beschreibung:

Verknüpfen mehrere Bedingungen.

Operator	Bedeutung	Beispiel	Ergebnis
&&	Und (AND)	`true && false`	`false`
||	Oder (OR)	`true || false`	`true`
!	Nicht (NOT)	`!true`	`false`

Beispiele:

```java
int alter = 20;
boolean hatFuehrerschein = true;

// UND: Beide Bedingungen muessen wahr sein
if (alter >= 18 && hatFuehrerschein) {
    System.out.println("Darf Auto fahren");
}

// ODER: Mindestens eine Bedingung muss wahr sein
int note = 1;
if (note == 1 || note == 2) {
    System.out.println("Gute Note!");
}

// NICHT: Kehrt Wahrheitswert um
boolean gefunden = false;
if (!gefunden) {
    System.out.println("Noch nicht gefunden");
}

// Array-Grenzen pruefen
int[] array = {1, 2, 3};
int i = 1;
if (i >= 0 && i < array.length) {
    System.out.println(array[i]);
}

// Zeichen ist Buchstabe oder Ziffer
char c = 'A';
if ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')) {
    System.out.println("Ist ein Buchstabe");
}
```

Kurzschluss-Auswertung:

```
// Bei && wird rechts nicht mehr ausgewertet, wenn links schon false ist
int[] zahlen = {1, 2, 3};
int index = 5;
if (index < zahlen.length && zahlen[index] > 0) {
    // Kein Fehler! zahlen[index] wird nicht ausgefuehrt, da index < zahlen
    .length bereits false ist
}
```

Zuweisungsoperatoren

Beschreibung:

Weisen einer Variable einen Wert zu oder ändern ihn.

Operator	Bedeutung	Beispiel	Entspricht
=	Zuweisung	a = 5	-
+=	Addition und Zuweisung	a += 3	a = a + 3
-=	Subtraktion und Zuweisung	a -= 3	a = a - 3
*=	Multiplikation und Zuweisung	a *= 3	a = a * 3
/=	Division und Zuweisung	a /= 3	a = a / 3
%=	Modulo und Zuweisung	a %= 3	a = a % 3

Beispiele:

```
int x = 10;
x += 5;   // x = 15 (entspricht: x = x + 5)
x -= 3;   // x = 12 (entspricht: x = x - 3)
x *= 2;   // x = 24 (entspricht: x = x * 2)
x /= 4;   // x = 6  (entspricht: x = x / 4)
x %= 4;   // x = 2  (entspricht: x = x % 4)

// Summe aufaddieren
int summe = 0;
int[] zahlen = {5, 10, 15};
for (int i = 0; i < zahlen.length; i++) {
    summe += zahlen[i];  // Kuerzer als: summe = summe + zahlen[i]
}

// Zeichen zaehlen
String text = "Hallo";
int anzahl = 0;
for (int i = 0; i < text.length(); i++) {
    if (text.charAt(i) == 'l') {
        anzahl++;  // Entspricht: anzahl = anzahl + 1
    }
}
```

Inkrement- und Dekrement-Operatoren

Beschreibung:

Erhöhen oder verringern einen Wert um 1.

Operator	Bedeutung	Beispiel	Ergebnis
++	Inkrement (erhöhen um 1)	`i++`	`i = i + 1`
-	Dekrement (verringern um 1)	`i-`	`i = i - 1`

Beispiele:

```
int i = 5;
i++;        // i = 6 (entspricht: i = i + 1)
i--;        // i = 5 (entspricht: i = i - 1)

// In Schleifen
for (int j = 0; j < 5; j++) {  // j wird nach jedem Durchlauf um 1 erhoeht
    System.out.println(j);
}

// Countdown
int countdown = 10;
while (countdown > 0) {
    System.out.println(countdown);
    countdown--;
}

// Zaehler erhoehen
int anzahl = 0;
String text = "Hallo";
for (int k = 0; k < text.length(); k++) {
    if (text.charAt(k) == 'l') {
        anzahl++;
    }
}
```

Unterschied: Prä- vs. Post-Inkrement:

```
int a = 5;
int b = a++;  // b = 5, dann a = 6 (Post-Inkrement: erst zuweisen, dann
    erhoehen)

int c = 5;
int d = ++c;  // c = 6, dann d = 6 (Prae-Inkrement: erst erhoehen, dann
    zuweisen)
```

String-Konkatenation

Beschreibung:

Der + Operator verbindet Strings.

Beispiele:

```
String vorname = "Anna";
String nachname = "Schmidt";
String vollname = vorname + " " + nachname;  // "Anna Schmidt"

// Mit Zahlen
int alter = 25;
String text = "Ich bin " + alter + " Jahre alt";  // "Ich bin 25 Jahre alt"

// In Schleifen
String wort = "Hallo";
String umgekehrt = "";
for (int i = wort.length() - 1; i >= 0; i--) {
    umgekehrt = umgekehrt + wort.charAt(i);
}
// umgekehrt = "ollaH"

// Array-Elemente verbinden
int[] zahlen = {1, 2, 3, 4, 5};
String ausgabe = "";
for (int i = 0; i < zahlen.length; i++) {
    ausgabe += zahlen[i];
    if (i < zahlen.length - 1) {
        ausgabe += ", ";
    }
}
// ausgabe = "1, 2, 3, 4, 5"
```

Operatorrangfolge (Priorität)

Beschreibung:

Operatoren werden in einer bestimmten Reihenfolge ausgewertet.

Von höchster zu niedrigster Priorität:

1. **Klammern**: ( )
2. **Inkrement/Dekrement**: ++, -
3. **Negation**: !
4. **Multiplikation, Division, Modulo**: *, /, %
5. **Addition, Subtraktion**: +, -
6. **Vergleichsoperatoren**: <, >, <=, >=
7. **Gleichheit**: ==, !=
8. **Logisches UND**: &&
9. **Logisches ODER**: ||
10. **Zuweisung**: =, +=, -=, etc.

Beispiele:

```
int ergebnis = 5 + 3 * 2;      // ergebnis = 11 (nicht 16!)
// * wird vor + ausgefuehrt: 5 + (3 * 2) = 5 + 6 = 11

int x = (5 + 3) * 2;           // x = 16
// Klammern aendern die Reihenfolge: (5 + 3) * 2 = 8 * 2 = 16

boolean b = 5 > 3 && 2 < 4;    // b = true
// Erst Vergleiche, dann &&: (5 > 3) && (2 < 4) = true && true = true

int y = 10 + 5 * 2 - 3;        // y = 17
// Reihenfolge: 5 * 2 = 10, dann 10 + 10 = 20, dann 20 - 3 = 17

// Bei Unsicherheit: Klammern verwenden!
int z = ((10 + 5) * 2) - 3;    // z = 27 (macht Absicht deutlich)
```

Worauf achten:

- Im Zweifelsfall immer Klammern setzen, um die Absicht klar zu machen!
- Multiplikation und Division haben höhere Priorität als Addition und Subtraktion
- Vergleichsoperatoren werden vor logischen Operatoren ausgewertet
- Klammern haben immer höchste Priorität

Kontrollstrukturen in Java

If-Bedingungen

Warum braucht man If-Bedingungen?

If-Bedingungen ermöglichen es, Code nur unter bestimmten Voraussetzungen auszuführen. Sie sind notwendig, wenn das Programm Entscheidungen treffen soll: "Wenn eine Bedingung erfüllt ist, dann führe diese Anweisungen aus."

Typische Anwendungsfälle:

- Prüfen, ob ein Wert einen bestimmten Zustand hat
- Vergleichen von Werten
- Reagieren auf Benutzereingaben
- Fehlerbehandlung (z.B. Index-Prüfung)

Einfache If-Bedingung

Syntax:

```
if (Bedingung) {
    Anweisung;
}
```

Beispiele:

```
// Zahl pruefen
int alter = 18;
if (alter >= 18) {
    System.out.println("Volljaehrig");
}

// Array-Index pruefen
int[] zahlen = {10, 20, 30};
int index = 1;
if (index >= 0 && index < zahlen.length) {
    System.out.println(zahlen[index]);  // Sicher zugreifen
}

// Zeichen pruefen
String text = "Hallo";
if (text.charAt(0) == 'H') {
    System.out.println("Beginnt mit H");
}

// String-Laenge pruefen
String wort = "Test";
if (wort.length() > 3) {
    System.out.println("Wort ist laenger als 3 Zeichen");
}

// Wert gefunden
boolean gefunden = false;
int[] liste = {5, 10, 15, 20};
int gesucht = 15;
for (int i = 0; i < liste.length; i++) {
    if (liste[i] == gesucht) {
        gefunden = true;
    }
}
```

If-Else-Bedingung

Syntax:

```
if (Bedingung) {
    Anweisung1;
} else {
    Anweisung2;
}
```

Beispiele:

```
// Gerade oder ungerade
int zahl = 7;
if (zahl % 2 == 0) {
    System.out.println("Gerade");
} else {
    System.out.println("Ungerade");
}

// Positiv oder negativ
int wert = -5;
if (wert >= 0) {
    System.out.println("Positiv oder Null");
} else {
    System.out.println("Negativ");
}

// Array leer oder gefuellt
int[] zahlen = {1, 2, 3};
if (zahlen.length > 0) {
    System.out.println("Array hat " + zahlen.length + " Elemente");
} else {
    System.out.println("Array ist leer");
}

// Zeichen vorhanden
String text = "Java";
if (text.length() > 0) {
    char erstes = text.charAt(0);
    System.out.println("Erstes Zeichen: " + erstes);
} else {
    System.out.println("String ist leer");
}
```

If-Else-If-Bedingung

Syntax:

```
if (Bedingung1) {
    Anweisung1;
} else if (Bedingung2) {
    Anweisung2;
} else {
    Anweisung3;
}
```

Beispiele:

```
// Notenbewertung
int punkte = 85;
if (punkte >= 90) {
    System.out.println("Note: Sehr gut");
} else if (punkte >= 80) {
    System.out.println("Note: Gut");
} else if (punkte >= 70) {
    System.out.println("Note: Befriedigend");
} else {
    System.out.println("Note: Nicht bestanden");
}

// Groessenvergleich
int a = 5;
int b = 10;
if (a > b) {
    System.out.println("a ist groesser");
} else if (a < b) {
    System.out.println("b ist groesser");
} else {
    System.out.println("a und b sind gleich");
}

// String-Laenge kategorisieren
String eingabe = "Hallo";
if (eingabe.length() == 0) {
    System.out.println("Leer");
} else if (eingabe.length() < 5) {
    System.out.println("Kurz");
} else if (eingabe.length() < 10) {
    System.out.println("Mittel");
} else {
```

```
33     System.out.println("Lang");
34 }
35
36 // Index-Position bestimmen
37 int[] werte = {10, 20, 30, 40, 50};
38 int index = 2;
39 if (index < 0) {
40     System.out.println("Index zu klein");
41 } else if (index >= werte.length) {
42     System.out.println("Index zu gross");
43 } else {
44     System.out.println("Wert: " + werte[index]);
45 }
```

Verschachtelte If-Bedingungen

Syntax:

```
if (Bedingung1) {
    if (Bedingung2) {
        Anweisung;
    }
}
```

Beispiele:

```
// Doppelte Pruefung
String text = "Test";
if (text.length() > 0) {
    if (text.charAt(0) == 'T') {
        System.out.println("Beginnt mit T und ist nicht leer");
    }
}

// Array-Zugriff mit Pruefung
int[] zahlen = {5, 10, 15, 20};
int index = 2;
if (index >= 0 && index < zahlen.length) {
    if (zahlen[index] > 10) {
        System.out.println("Wert an Position " + index + " ist groesser als
    10");
    }
}

// Mehrere Bedingungen
int zahl = 24;
if (zahl > 0) {
    if (zahl % 2 == 0) {
        if (zahl < 100) {
            System.out.println("Positive gerade Zahl unter 100");
        }
    }
}
```

For-Schleifen

Warum braucht man For-Schleifen?

For-Schleifen ermöglichen es, Code mehrfach auszuführen. Sie sind besonders nützlich, wenn man:

- Alle Elemente eines Arrays durchlaufen möchte
- Alle Zeichen eines Strings prüfen will
- Eine Aufgabe eine bestimmte Anzahl wiederholen möchte
- Mit Indizes arbeiten muss (z.B. zum Ändern von Array-Werten)

Standard For-Schleife

Syntax:

```
for (Initialisierung; Bedingung; Inkrement) {
    Anweisung;
}
```

Wichtig:

Alle drei Teile (Initialisierung, Bedingung, Inkrement) sind **optional** und können weggelassen werden!

Typisches Muster für Arrays:

```
for (int i = 0; i < array.length; i++) {
    // Verwende array[i]
}
```

Typisches Muster für Strings:

```
for (int i = 0; i < text.length(); i++) {
    // Verwende text.charAt(i)
}
```

Beispiele:

```
// Zahlen von 0 bis 4 ausgeben
for (int i = 0; i < 5; i++) {
    System.out.println(i);  // Gibt aus: 0, 1, 2, 3, 4
}

// Array durchlaufen
int[] zahlen = {10, 20, 30, 40, 50};
for (int i = 0; i < zahlen.length; i++) {
    System.out.println(zahlen[i]);
}

// String durchlaufen
String text = "Hallo";
for (int i = 0; i < text.length(); i++) {
    char c = text.charAt(i);
    System.out.println(c);
}
```

```java

// Array-Werte aendern
int[] werte = {1, 2, 3, 4, 5};
for (int i = 0; i < werte.length; i++) {
    werte[i] = werte[i] * 2;  // Jeden Wert verdoppeln
}
// Ergebnis: {2, 4, 6, 8, 10}

// Array befuellen
int[] quadrate = new int[5];
for (int i = 0; i < quadrate.length; i++) {
    quadrate[i] = i * i;
}
// Ergebnis: {0, 1, 4, 9, 16}

// Summe berechnen
int[] liste = {5, 10, 15, 20};
int summe = 0;
for (int i = 0; i < liste.length; i++) {
    summe = summe + liste[i];
}
System.out.println("Summe: " + summe);  // Summe: 50

// Zeichen zaehlen
String wort = "Banane";
int anzahlA = 0;
for (int i = 0; i < wort.length(); i++) {
    if (wort.charAt(i) == 'a') {
        anzahlA++;
    }
}
System.out.println("Anzahl 'a': " + anzahlA);  // Anzahl 'a': 3

// Rueckwaerts durchlaufen
int[] zahlen2 = {1, 2, 3, 4, 5};
for (int i = zahlen2.length - 1; i >= 0; i--) {
    System.out.println(zahlen2[i]);  // Gibt aus: 5, 4, 3, 2, 1
}

// Nur gerade Indizes
int[] daten = {10, 20, 30, 40, 50, 60};
for (int i = 0; i < daten.length; i = i + 2) {
    System.out.println(daten[i]);  // Gibt aus: 10, 30, 50
}

// Teilstring durchsuchen (von Position 2 bis Ende)
String satz = "Hallo Welt";
```

```
for (int i = 2; i < satz.length(); i++) {
    System.out.println(satz.charAt(i));  // Gibt aus: l, l, o, , W, e, l, t
}
```

Optionale Teile der For-Schleife

Alle drei Teile können weggelassen werden!

Initialisierung weglassen

Wenn die Variable bereits vorher deklariert wurde:

```
// Beispiel 1: Zaehler bereits vorhanden
int i = 0;
for (; i < 5; i++) {
    System.out.println(i);
}

// Beispiel 2: Start-Index von aussen vorgegeben
int startIndex = 2;
for (; startIndex < 10; startIndex++) {
    System.out.println(startIndex);
}
```

Bedingung weglassen

Wenn die Bedingung immer wahr sein soll (Endlosschleife oder mit `break` beenden):

```
// Beispiel 1: Endlosschleife mit break
int[] zahlen = {10, 20, 30, 40, 50};
for (int i = 0; ; i++) {
    if (i >= zahlen.length) {
        break;  // Schleife manuell beenden
    }
    System.out.println(zahlen[i]);
}

// Beispiel 2: Suche mit vorzeitigem Abbruch
String text = "Hallo Welt";
for (int i = 0; ; i++) {
    if (i >= text.length()) {
        break;
    }
    if (text.charAt(i) == 'W') {
        System.out.println("Gefunden an Position: " + i);
        break;
    }
}
```

Inkrement weglassen

Wenn die Erhöhung manuell im Schleifenkörper erfolgt:

```java
// Beispiel 1: Variable Schrittweite
int[] zahlen = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
for (int i = 0; i < zahlen.length; ) {
    System.out.println(zahlen[i]);
    if (zahlen[i] % 2 == 0) {
        i = i + 1;  // Nach geraden Zahlen: 1 Schritt
    } else {
        i = i + 2;  // Nach ungeraden Zahlen: 2 Schritte
    }
}

// Beispiel 2: Bedingte Erhoehung
String text = "aabbbcccc";
for (int i = 0; i < text.length(); ) {
    char c = text.charAt(i);
    System.out.println("Zeichen: " + c);
    // Ueberspringe alle gleichen Zeichen
    while (i < text.length() && text.charAt(i) == c) {
        i++;  // Erhoehung hier statt im for-Kopf
    }
}
```

Mehrere Teile weglassen

```java
// Alle Teile weggelassen = Endlosschleife
// for (;;) {
//     System.out.println("Laeuft unendlich");
// }

// Nur Bedingung vorhanden
int i = 0;
for (; i < 5; ) {
    System.out.println(i);
    i++;
}
```

Verschachtelte For-Schleifen

Syntax:

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        Anweisung;
    }
}
```

Beispiele:

```
// Multiplikationstabelle
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        System.out.println(i + " * " + j + " = " + (i * j));
    }
}

// Zwei Arrays vergleichen
int[] array1 = {1, 2, 3};
int[] array2 = {2, 3, 4};
for (int i = 0; i < array1.length; i++) {
    for (int j = 0; j < array2.length; j++) {
        if (array1[i] == array2[j]) {
            System.out.println("Gemeinsamer Wert: " + array1[i]);
        }
    }
}

// String-Vergleich (naiv)
String text1 = "abc";
String text2 = "bcd";
for (int i = 0; i < text1.length(); i++) {
    for (int j = 0; j < text2.length(); j++) {
        if (text1.charAt(i) == text2.charAt(j)) {
            System.out.println("Gemeinsames Zeichen: " + text1.charAt(i));
        }
    }
}

// Pattern ausgeben
for (int i = 1; i <= 4; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("*");
    }
```

```
35     System.out.println();
36 }
37 // Ausgabe:
38 // *
39 // **
40 // ***
41 // ****
```

While-Schleifen

Warum braucht man While-Schleifen?

While-Schleifen verwendet man, wenn:

- Die Anzahl der Wiederholungen im Voraus nicht bekannt ist
- Die Schleife bis zum Eintreten einer bestimmten Bedingung laufen soll
- Man “so lange wie nötig“ wiederholen möchte
- Man nach einem bestimmten Element sucht und abbrechen will, wenn es gefunden wurde

Unterschied zur For-Schleife:

- For-Schleife: “Wiederhole genau n-mal“ oder “Durchlaufe alle Elemente“
- While-Schleife: “Wiederhole solange diese Bedingung wahr ist“

While-Schleife

Syntax:

```
while (Bedingung) {
    Anweisung;
}
```

Beispiele:

```
// Zahlen von 0 bis 4 ausgeben
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;
}

// Element in Array suchen
int[] zahlen = {10, 20, 30, 40, 50};
int gesucht = 30;
int index = 0;
boolean gefunden = false;
while (index < zahlen.length && !gefunden) {
    if (zahlen[index] == gesucht) {
        gefunden = true;
        System.out.println("Gefunden an Position: " + index);
    }
    index++;
}

// Erste Position eines Zeichens finden
String text = "Hallo Welt";
char gesuchtesZeichen = 'W';
int position = 0;
while (position < text.length()) {
    if (text.charAt(position) == gesuchtesZeichen) {
        System.out.println("Gefunden an Position: " + position);
        break;  // Schleife beenden
    }
    position++;
}

// Solange Wert unter Grenze
int summe = 0;
int zahl = 1;
while (summe < 100) {
```

```
37      summe = summe + zahl;
38      zahl++;
39  }
40  System.out.println("Summe: " + summe);
41  System.out.println("Zahlen addiert: " + (zahl - 1));
42
43  // Array durchlaufen (alternative zu for)
44  int[] werte = {5, 10, 15, 20};
45  int idx = 0;
46  while (idx < werte.length) {
47      System.out.println(werte[idx]);
48      idx++;
49  }
50
51  // Bedingung wird nie erfuellt -> Schleife laeuft nicht
52  int x = 10;
53  while (x < 5) {
54      System.out.println("Wird nicht ausgegeben");
55      x++;
56  }
57
58  // Countdown
59  int countdown = 5;
60  while (countdown > 0) {
61      System.out.println(countdown);
62      countdown--;
63  }
64  System.out.println("Start!");
65
66  // Erstes Vorkommen eines Teilstrings finden
67  String satz = "Der Hund spielt im Garten";
68  String suche = "spielt";
69  int pos = 0;
70  boolean gefundenString = false;
71  while (pos <= satz.length() - suche.length() && !gefundenString) {
72      boolean match = true;
73      for (int j = 0; j < suche.length(); j++) {
74          if (satz.charAt(pos + j) != suche.charAt(j)) {
75              match = false;
76          }
77      }
78      if (match) {
79          gefundenString = true;
80          System.out.println("Gefunden an Position: " + pos);
81      }
82      pos++;
83  }
```

Vergleich: For vs. While

Wann For-Schleife?

```
// Wenn die Anzahl der Durchlaeufe bekannt ist
for (int i = 0; i < 10; i++) { }

// Beim Durchlaufen von Arrays
int[] zahlen = {1, 2, 3, 4, 5};
for (int i = 0; i < zahlen.length; i++) { }

// Beim Durchlaufen von Strings
String text = "Hallo";
for (int i = 0; i < text.length(); i++) { }

// Wenn man den Index braucht
for (int i = 0; i < array.length; i++) {
    array[i] = i * 2;  // Index wird zum Aendern gebraucht
}
```

Wann While-Schleife?

```
// Wenn die Anzahl der Durchlaeufe unbekannt ist
while (summe < 1000) {
    summe = summe + wert;
}

// Bei Suche mit vorzeitigem Abbruch
while (index < array.length && !gefunden) {
    if (array[index] == gesucht) {
        gefunden = true;
    }
    index++;
}

// Bedingungsabhaengige Wiederholung
while (bedingungErfuellt()) {
    macheEtwas();
}
```

Häufige Fehler

Endlosschleifen

```
// FALSCH - Endlosschleife!
int i = 0;
while (i < 5) {
    System.out.println(i);
    // i++ vergessen! Schleife laeuft unendlich
}

// RICHTIG
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;  // Zaehler erhoehen
}
```

Falsche Bedingungen

```
// FALSCH - ArrayIndexOutOfBoundsException!
int[] zahlen = {1, 2, 3};
for (int i = 0; i <= zahlen.length; i++) {  // <= ist falsch!
    System.out.println(zahlen[i]);
}

// RICHTIG
for (int i = 0; i < zahlen.length; i++) {  // < ist richtig
    System.out.println(zahlen[i]);
}
```

Semikolon-Fehler

```
// FALSCH - Semikolon beendet Schleife sofort
for (int i = 0; i < 5; i++);  // Semikolon hier ist falsch!
{
    System.out.println(i);  // Wird nur einmal ausgefuehrt, i nicht bekannt
}

// RICHTIG
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

Off-by-One Fehler

```
String text = "Hallo";  // length = 5, Indizes: 0-4

// FALSCH
for (int i = 0; i <= text.length(); i++) {  // i = 5 ist zu viel!
    System.out.println(text.charAt(i));
}

// RICHTIG
for (int i = 0; i < text.length(); i++) {
    System.out.println(text.charAt(i));
}

// RICHTIG - Letztes Element direkt
char letztes = text.charAt(text.length() - 1);
```

Grundlegende String-Methoden in Java

String-Methoden

length() - Länge eines Strings

Was macht die Methode?

Gibt die Anzahl der Zeichen im String zurück.

Syntax:

```
int laenge = text.length();
```

Beispiele:

```
String name = "Anna";
int laenge = name.length();  // laenge = 4

String wort = "Hallo Welt";
int laenge2 = wort.length();  // laenge2 = 10 (Leerzeichen zaehlt mit!)

String leer = "";
int laenge3 = leer.length();  // laenge3 = 0

String satz = "Java";
for (int i = 0; i < satz.length(); i++) {
    // Schleife laeuft 4 mal (von 0 bis 3)
}
```

Worauf achten:

- Die Zählung der Zeichen beginnt bei 1 (erster Buchstabe = Position 0, aber length = 1)
- Leerzeichen und Sonderzeichen zählen mit
- Bei Strings: `.length()` **mit Klammern**
- Bei Arrays: `.length` **ohne Klammern**

charAt(int index) - Zeichen an Position

Was macht die Methode?

Gibt das Zeichen an der angegebenen Position zurück. Die Zählung beginnt bei 0!

Syntax:

```
char zeichen = text.charAt(index);
```

Beispiele:

```
String wort = "Hallo";
char erstes = wort.charAt(0);   // erstes = 'H'
char zweites = wort.charAt(1);  // zweites = 'a'
char letztes = wort.charAt(4);  // letztes = 'o'

// Alle Zeichen durchlaufen
String text = "Java";
for (int i = 0; i < text.length(); i++) {
    char c = text.charAt(i);
    System.out.println(c);  // Gibt aus: J, a, v, a
}

// Pruefung einzelner Zeichen
String eingabe = "A1B2";
if (eingabe.charAt(0) == 'A') {
    System.out.println("Beginnt mit A");
}

// Zeichen vergleichen
String code = "12345";
if (code.charAt(2) == '3') {
    System.out.println("An Position 2 steht eine 3");
}
```

Worauf achten:

- Index beginnt bei 0
- Index muss zwischen 0 und `text.length() - 1` liegen
- Bei ungültigem Index: `StringIndexOutOfBoundsException`
- Gibt ein `char` zurück, keinen `String`

Häufige Fehler bei Strings

String.length ohne Klammern

```
// FALSCH
String text = "Hallo";
int laenge = text.length;  // Fehler: Strings brauchen ()

// RICHTIG
int laenge = text.length();    // String MIT Klammern
```

Index-Grenzen bei charAt

```
String wort = "Test";  // length = 4, Indizes: 0, 1, 2, 3
char c = wort.charAt(4);  // Fehler! Index 4 existiert nicht

// RICHTIG
char letztes = wort.charAt(wort.length() - 1);  // Index 3
char erstes = wort.charAt(0);                   // Index 0
```

char vs. String

```
String text = "A";
char buchstabe = 'A';

// FALSCH
if (text.charAt(0) == "A") { }  // Vergleicht char mit String

// RICHTIG
if (text.charAt(0) == 'A') { }  // char mit char vergleichen
```